

23-24: Instalación de XV6 sobre kali-linux mediante Docker
Comando hello
Compilación

Instalación de XV6. Hello World.

- Crear un directorio de trabajo XV6_Dev con el comando

`mkdir XV6_dev` 23-24: Misma carpeta, XV6_dev

- Desde dentro de ese directorio, clonar xv6 desde github

`git clone git://github.com/mit-pdos/xv6-public.git`

 ma@LAPTOP-0VBOP51T: ~/XV6_dev 23-24: git clone https://github.com/mit-pdos/xv6-public.git

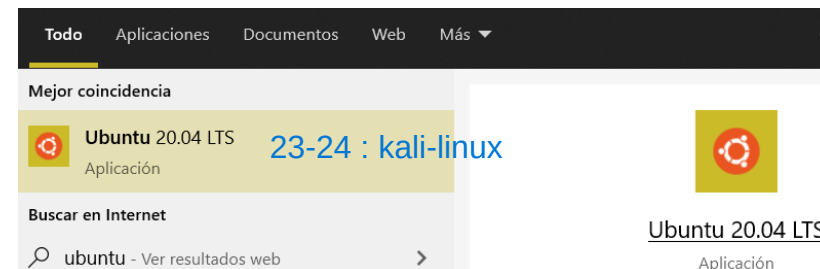
```
ma@LAPTOP-0VBOP51T:~$ mkdir XV6_dev
ma@LAPTOP-0VBOP51T:~$ cd XV6_dev/
ma@LAPTOP-0VBOP51T:~/XV6_dev$ git clone git://github.com/mit-pdos/xv6-public.git
Cloning into 'xv6-public'...
remote: Enumerating objects: 13990, done.
remote: Total 13990 (delta 0), reused 0 (delta 0), pack-reused 13990
Receiving objects: 100% (13990/13990), 17.18 MiB | 5.11 MiB/s, done.
Resolving deltas: 100% (9538/9538), done.
ma@LAPTOP-0VBOP51T:~/XV6_dev$ _
```

- Ponemos en ejecución el Docker Daemon.

Primero, debemos tener el Docker daemon corriendo `sudo dockerd`

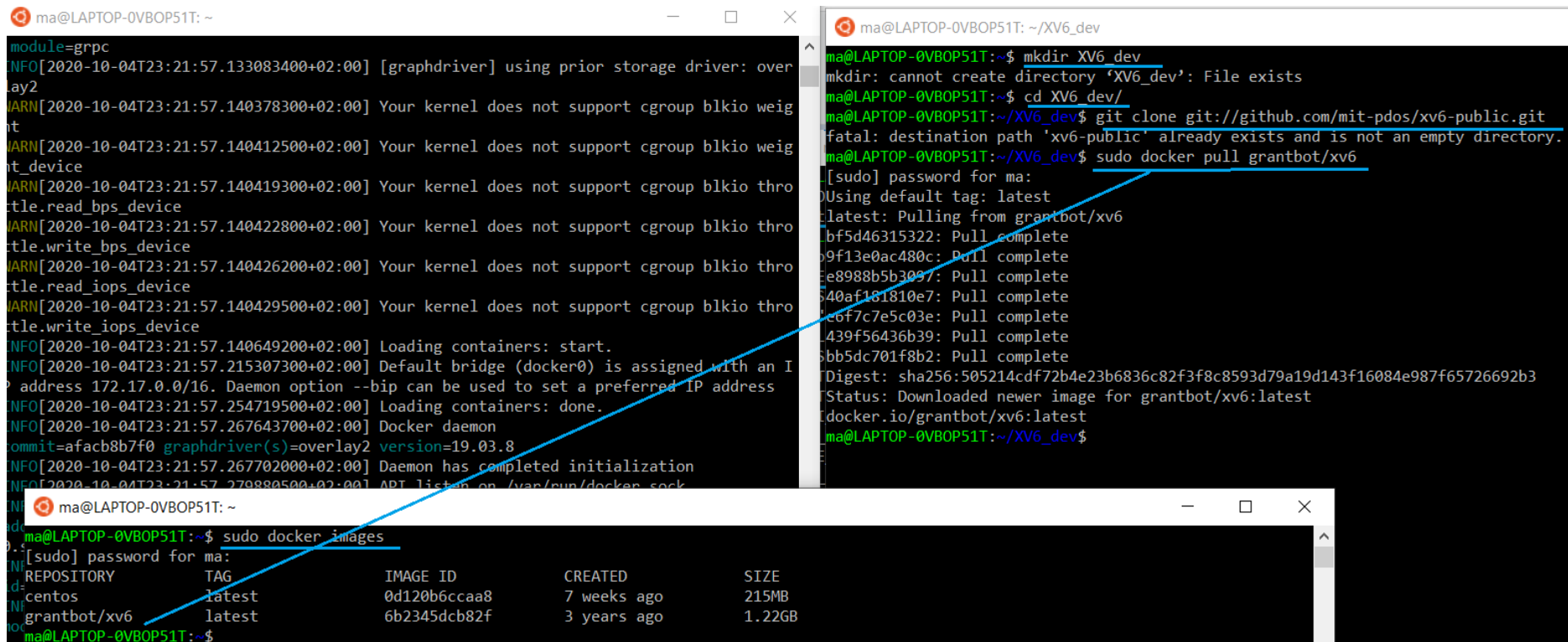
```
ma@LAPTOP-0VBOP51T: ~/XV6_dev
ma@LAPTOP-0VBOP51T:~/XV6_dev$ sudo dockerd
[sudo] password for ma:
INFO[2021-09-26T21:08:28.979373700+02:00] Starting up
INFO[2021-09-26T21:08:28.984946500+02:00] libcontainerd: started new containerd
INFO[2021-09-26T21:08:28.985030800+02:00] parsed scheme: "unix"
INFO[2021-09-26T21:08:28.985078900+02:00] scheme "unix" not registered, fallback to default scheme
INFO[2021-09-26T21:08:28.985125400+02:00] ccResolverWrapper: sending update to ccResolverImpl
INFO[2021-09-26T21:08:28.985234100+02:00] ClientConn switching balancer to "pickfirst"
WARN[0000] deprecated version : `1`, please switch to version `2`
INFO[2021-09-26T21:08:29.137594400+02:00] starting containerd
0ubuntu1~20.04.2"
INFO[2021-09-26T21:08:29.159323000+02:00] loading plugin "io.containerd.contentd.content.v1"
INFO[2021-09-26T21:08:29.162156000+02:00] loading plugin "io.containerd.snapshotter.v1"
INFO[2021-09-26T21:08:29.177098700+02:00] skip loading plugin "io.containerd.grpc.v1"
is not supported (modprobe aufs failed: exit status 1 \ "modprobe: FATAL: Mod
```

Ésta será la Terminal1, abrimos otra...



- Instalar la imagen de xv6 en docker [Lleva tiempo] (en otra terminal)

`sudo docker pull grantbot/xv6`



```
ma@LAPTOP-0VBOP51T: ~  
module=grpc  
INFO[2020-10-04T23:21:57.133083400+02:00] [graphdriver] using prior storage driver: overlay2  
WARN[2020-10-04T23:21:57.140378300+02:00] Your kernel does not support cgroup blkio weight  
WARN[2020-10-04T23:21:57.140412500+02:00] Your kernel does not support cgroup blkio weight  
WARN[2020-10-04T23:21:57.140419300+02:00] Your kernel does not support cgroup blkio throttle.read_bps_device  
WARN[2020-10-04T23:21:57.140422800+02:00] Your kernel does not support cgroup blkio throttle.write_bps_device  
WARN[2020-10-04T23:21:57.140426200+02:00] Your kernel does not support cgroup blkio throttle.read_iops_device  
WARN[2020-10-04T23:21:57.140429500+02:00] Your kernel does not support cgroup blkio throttle.write_iops_device  
INFO[2020-10-04T23:21:57.140649200+02:00] Loading containers: start.  
INFO[2020-10-04T23:21:57.215307300+02:00] Default bridge (docker0) is assigned with an IP address 172.17.0.0/16. Daemon option --bip can be used to set a preferred IP address  
INFO[2020-10-04T23:21:57.254719500+02:00] Loading containers: done.  
INFO[2020-10-04T23:21:57.267643700+02:00] Docker daemon  
commit=afacb8b7f0 graphdriver(s)=overlay2 version=19.03.8  
INFO[2020-10-04T23:21:57.267702000+02:00] Daemon has completed initialization  
INFO[2020-10-04T23:21:57.270880500+02:00] API listen on /var/run/docker.sock  
ma@LAPTOP-0VBOP51T: ~  
ma@LAPTOP-0VBOP51T:~$ sudo docker images  
[sudo] password for ma:  
REPOSITORY          TAG             IMAGE ID        CREATED         SIZE  
centos               latest          0d120b6ccaa8    7 weeks ago    215MB  
grantbot/xv6         latest         6b2345dcb82f    3 years ago    1.22GB  
ma@LAPTOP-0VBOP51T:~$
```

```
ma@LAPTOP-0VBOP51T: ~/XV6_dev  
ma@LAPTOP-0VBOP51T:~$ mkdir XV6_dev  
mkdir: cannot create directory 'XV6_dev': File exists  
ma@LAPTOP-0VBOP51T:~$ cd XV6_dev/  
ma@LAPTOP-0VBOP51T:~/XV6_dev$ git clone git://github.com/mit-pdos/xv6-public.git  
fatal: destination path 'xv6-public' already exists and is not an empty directory.  
ma@LAPTOP-0VBOP51T:~/XV6_dev$ sudo docker pull grantbot/xv6  
[sudo] password for ma:  
Using default tag: latest  
latest: Pulling from grantbot/xv6  
bf5d46315322: Pull complete  
9f13e0ac480c: Pull complete  
e8988b5b3097: Pull complete  
540af181810e7: Pull complete  
e6f7c7e5c03e: Pull complete  
439f56436b39: Pull complete  
5bb5dc701f8b2: Pull complete  
Digest: sha256:505214cdf72b4e23b6836c82f3f8c8593d79a19d143f16084e987f65726692b3  
Status: Downloaded newer image for grantbot/xv6:latest  
docker.io/grantbot/xv6:latest  
ma@LAPTOP-0VBOP51T:~/XV6_dev$
```

- Comprobamos las imágenes de Docker (en otra terminal)

`sudo docker images`

- Levantamos el contenedor XV6

sustituyendo {Path to local XV6 folder} por el path donde se ha instalado XV6_Dev

```
sudo docker run -v '{Path to local XV6 folder}/XV6_Dev':/home/a/XV6_Dev/ -i -t grantbot/xv6
```

Obs: en mi caso , mi usuario es ma, ver en la imagen siguiente.

Cuando se ejecuta el contenedor, el usuario es **a** (la **password también es a**). En otra terminal, se pueden ver los procesos docker corriendo.

Hay que asegurarse que el **servidor docker is running** (tenedlo abierto en otra terminal).

```
a@4e6b41157d30: ~/xv6-public  
ma@LAPTOP-0VBOP51T:~/XV6_dev$ sudo docker run -v '/home/ma/XV6_dev':/home/a/XV6_dev/ -i -t grantbot/xv6  
a@4e6b41157d30:/$ ls  
bin boot dev etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var  
a@4e6b41157d30:/$ cd home/a  
a@4e6b41157d30:~$ ls  
XV6_dev deps xv6-public  
a@4e6b41157d30:~$ cd xv6-public/  
a@4e6b41157d30:~/xv6-public$ cp wc.c hello.c_
```

Para crear nuestro comando hello copiamos un fichero fuente y después lo modificaremos con el editor nano.

- Instalación de nano dentro del contenedor

```
@c474f89b4543:~/xv6-public$ sudo apt install nano
Reading package lists... Done
Building dependency tree
Reading state information... Done
Suggested packages:
  spell
The following NEW packages will be installed:
  nano
0 upgraded, 1 newly installed, 0 to remove and 5 not upgraded.
Need to get 194 kB of archives.
After this operation, 614 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu/ trusty/main nano amd64 2.2.6-1ubuntu1 [194 kB]
Fetched 194 kB in 1s (156 kB/s)
Selecting previously unselected package nano.
Reading database ... 26482 files and directories currently installed.)
Preparing to unpack .../nano_2.2.6-1ubuntu1_amd64.deb ...
Unpacking nano (2.2.6-1ubuntu1) ...
Processing triggers for man-db (2.6.7.1-1ubuntu1) ...
Setting up nano (2.2.6-1ubuntu1) ...
update-alternatives: using /bin/nano to provide /usr/bin/editor (editor) in auto mode
update-alternatives: using /bin/nano to provide /usr/bin/pico (pico) in auto mode
@c474f89b4543:~/xv6-public$ nano hello.c
@c474f89b4543:~/xv6-public$ cat hello.c
#include "types.h"
#include "stat.h"
#include "user.h"

int
main(int argc, char *argv[])

    printf(1, "Hello World!\n");
    exit();
```

- Editamos hello.c mediante nano hello.c

```
a@a43076991834: ~/xv6-public
GNU nano 2.2.6 File: hello.c

#include "types.h"
#include "stat.h"
#include "user.h"

int main(int argc, char *argv[]){
    printf(1, "Hello World!, command provided by INSO SIOP C Team\n");
    exit();
}
```

```
a@a43076991834:~/xv6-public$ nano hello.c
a@a43076991834:~/xv6-public$ cat hello.c
#include "types.h"
#include "stat.h"
#include "user.h"

int main(int argc, char *argv[]){
    printf(1, "Hello World!, command provided by INSO SIOP C Team\n");
    exit();
}
a@a43076991834:~/xv6-public$
```

- Modificación de la utilidad de compilación make (archivo Makefile)
nano Makefile

a@c474f89b4543: ~/xv6-public

```
Makefile      cat.sym      gdbutil      kill.o        picirq.o      spinp         ulib.c
notes        console.c    grep.asm     kill.sym     pipe.c        stat.h        ulib.d
README       console.d    grep.c       lapic.c     pipe.d        stressfs.asm  ulib.o
TRICKS       console.o    grep.d       lapic.d     pipe.o        stressfs.c    umalloc.c
cat          cuth        grep.o       lapic.o     pr.pl         stressfs.d    umalloc.d
echo         date.h      grep.sym     ln.asm      printf.c      stressfs.o    umalloc.o
forktest     defs.h      hello.c     ln.c        printf.d      stressfs.sym  user.h
grep         dot-bochsrc ide.c        ln.d        printf.o      string.c      usertests.asm
init         echo.asm    ide.d        ln.o        printpcs     string.d      usertests.c
kill         echo.c      ide.o        ln.sym      proc.c       string.o      usertests.d
ln           echo.d      init.asm     log.c       proc.d       switch.S      usertests.o
ls           echo.o      init.c       log.d       proc.h       switch.o      usertests.sym
mkdir        echo.sym    init.d       log.o       proc.o       symlink.patch usys.S
rm           elf.h       init.o       ls.asm      rm.asm       syscall.c     usys.o
sh           entry.S     init.sym     ls.c        rm.c         syscall.d     vectors.S
stressfs     entry.o     initcode.S   ls.d        rm.d         syscall.h     vectors.o
usertests    entryother  initcode.asm ls.o        rm.o         syscall.o     vectors.pl
wc           entryother.S initcode.d    main.c      rm.sym       sysfile.c     vm.c
zombie       entryother.asm initcode.o    main.d      runoff       sysfile.d     vm.d
asm.h        entryother.d initcode.out  main.o      runoff.list  sysfile.o     vm.o
bio.c        entryother.o initcode.c    memide.c    runoff.spec  sysproc.c     wc.asm
bio.d        exec.c      ioapic.c     mkdir.asm   sh.asm       sysproc.d     wc.c
bio.o        exec.d      ioapic.d     mkdir.c     sh.c         timer.c       wc.d
bootasm.S    exec.o      ioapic.o     mkdir.d     sh.o         timer.d       wc.o
bootasm.d    fcntl.h    kalloc.c     mkdir.o     sh.sym       timer.o       wc.sym
bootasm.o    file.c     kalloc.d     mkdir.o     show1        toc.ftr       x86.h
bootblock    file.d     kalloc.o     mkdir.sym   sign.pl      toc.hdr       xv6.img
bootblock.asm file.h      kbd.c        mkfs        sleep1.p     trap.c        zombie.asm
bootblock.o  file.o     kbd.d        mmu.h       sleeplock.c  trap.d        zombie.c
bootblockother.o forktest.asm kbd.o        mp.c        sleeplock.d  trapasm.S     zombie.o
bootmain.c   forktest.c kbd.o        mp.d        sleeplock.h  trapasm.o     zombie.sym
bootmain.d   forktest.d kernel.asm   mp.h        sleeplock.o  traps.h
bootmain.o   forktest.o kernel.ld    mp.o        spinlock.c   types.h
cat.asm      fs.c       kernel.sym   param.h     spinlock.d   uart.c
cat.c        fs.d       kill.asm
a@c474f89b4543:~/xv6-public$ nano Makefile
```

- **UPROGS.** Adición de nuestro comando hello

```
a@a43076991834: ~/xv6-public
GNU nano 2.2.6 File: Makefile
# http://www.gnu.org/software/make/manual/html_node/Chai
.PRECIOUS: %.o

UPROGS=\
    _cat\
    _echo\
    _forktest\
    _grep\
    _init\
    _kill\
    _ln\
    _ls\
    _mkdir\
    _rm\
    _sh\
    _stressfs\
    _usertests\
    _wc\
    _zombie\
```

Añadimos una línea entre `_wc` y `_zombie` para que se compile nuestro comando hello

```
a@a43076991834: ~/xv6-public
GNU nano 2.2.6
# http://www.gnu.org/software/make/manual
.PRECIOUS: %.o

UPROGS=\
    _cat\
    _echo\
    _forktest\
    _grep\
    _init\
    _kill\
    _ln\
    _ls\
    _mkdir\
    _rm\
    _sh\
    _stressfs\
    _usertests\
    _wc\
    _hello\
    _zombie\

fs.img: mkfs README $(UPROGS)
./mkfs fs.img README $(UPROGS)
```


- EXTRA. Declaración de nuestro código fuente del comando hello

```

        else echo "-s -p $(GDBPORT)"; fi)
endif
CPUS := 2
endif
QEMUOPTS = -drive file=fs.img,index=1,media=disk,format=raw -drive file=xv6.img
smp $(CPUS) -m 512 $(QEMUEXTRA)

qemu: fs.img xv6.img
    $(QEMU) -serial mon:stdio $(QEMUOPTS)

qemu-memfs: xv6memfs.img
    $(QEMU) -drive file=xv6memfs.img,index=0,media=disk,format=raw -smp $(C

qemu-nox: fs.img xv6.img
    $(QEMU) -nographic $(QEMUOPTS)

.gdbinit: .gdbinit.tmpl
    sed "s/localhost:1234/localhost:$(GDBPORT)/" < $^ > $@

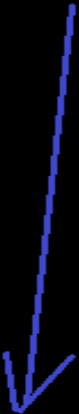
qemu-gdb: fs.img xv6.img .gdbinit
    @echo "**** Now run 'gdb'." 1>&2
    $(QEMU) -serial mon:stdio $(QEMUOPTS) -S $(QEMUGDB)

qemu-nox-gdb: fs.img xv6.img .gdbinit
    @echo "**** Now run 'gdb'." 1>&2
    $(QEMU) -nographic $(QEMUOPTS) -S $(QEMUGDB)

# CUT HERE
# prepare dist for students
# after running make dist, probably want to
# rename it to rev0 or rev1 or so on and then
# check in that version.

EXTRA=\
mkfs.c ulib.c user.h cat.c echo.c forktest.c grep.c kill.c\
ln.c ls.c mkdir.c rm.c stressfs.c usertests.c wc.c hello.c zombie.c\
printf.c umalloc.c\
README dot-bochsrc *.pl toc.* runoff runoff1 runoff.list\
.gdbinit.tmpl gdbutil\

```



- **Compilamos el sistema** `make qemu-nox`

```
a@a43076991834:~/xv6-public$ nano Makefile
a@a43076991834:~/xv6-public$ make qemu-nox
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD
-stack-protector -c -o hello.o hello.c
ld -m elf_i386 -N -e main -Ttext 0 -o _hello hello.o ulib.o usys.o
objdump -S _hello > hello.asm
objdump -t _hello | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > hello
./mkfs fs.img README _cat _echo _forktest _grep _init _kill _ln _ls _
zombie
nmeta 59 (boot, super, log blocks 30 inode blocks 26, bitmap blocks 1
ballocc: first 639 blocks have been allocated
ballocc: write bitmap block at sector 58
dd if=/dev/zero of=xv6.img count=10000
10000+0 records in
10000+0 records out
5120000 bytes (5.1 MB) copied, 0.0314972 s, 163 MB/s
dd if=bootblock of=xv6.img conv=notrunc
1+0 records in
1+0 records out
512 bytes (512 B) copied, 0.0010843 s, 472 kB/s
dd if=kernel of=xv6.img seek=1 conv=notrunc
354+1 records in
354+1 records out
181312 bytes (181 kB) copied, 0.0023028 s, 78.7 MB/s
qemu-system-i386 -nographic -drive file=fs.img,index=1,media=disk,for
format=raw -smp 2 -m 512
xv6...
```

- Una vez arrancado , el sistema contiene nuestro comando hello y podemos ejecutarlo.

```
1+0 records out
512 bytes (512 B) copied, 0.0001272 s, 4.0 MB/s
dd if=kernel of=xv6.img seek=1 conv=notrunc
354+1 records in
354+1 records out
181312 bytes (181 kB) copied, 0.0013524 s, 134 MB/s
qemu-system-i386 -nographic -drive file=fs.img,index=1,media=disk,format=raw -drive file=xv6.img,index=0,media=
disk,format=raw -smp 2 -m 512
xv6...
cpu1: starting
cpu0: starting
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap start 58
init: starting sh
$ ls
.          1 1 512
..         1 1 512
README    2 2 2487
cat        2 3 14368
echo       2 4 13233
forktest  2 5 8075
grep       2 6 15924
init       2 7 14118
kill       2 8 13277
ln         2 9 13199
ls         2 10 16055
mkdir     2 11 13298
rm         2 12 13275
sh         2 13 24735
stressfs  2 14 14201
usertests  2 15 67133
wc         2 16 15054
hello     2 17 13018
zombie    2 18 12943
console   3 19 0
$ hello
Hello World!
$
```

- Para salir.

- Ctrl+ a (ambas pulsadas), soltamos y pulsamos C
- quit
- exit

```

wc          2 16 15054
hello       2 17 13058
zombie      2 18 12943
console     3 19 0
$ hello
Hello World!, command provided by INSO SIOP C Team Ctrl + a, soltamos y pulsa c
$ QEMU 2.0.0 monitor - type 'help' for more information
(qemu) quit
a@a43076991834:~/xv6-public$ exit
exit
ma@LAPTOP-0VBOP51T:~/XV6_dev/xv6-public$

```

- Paramos también el servidor daemond

```

WARN[2021-09-26T21:50:53.140053600+02:00] cleanup warnings time="2021-09-26T21:50:53.140053600+02:00"
ing signal loop" namespace=moby pid=759
^CINFO[2021-09-26T21:53:50.314234200+02:00] Processing signal 'interrupt'
INFO[2021-09-26T21:53:50.315397600+02:00] stopping event stream following
=libcontainerd namespace=moby
INFO[2021-09-26T21:53:50.315860300+02:00] Daemon shutdown complete
INFO[2021-09-26T21:53:50.315962400+02:00] stopping healthcheck following g
INFO[2021-09-26T21:53:50.315965200+02:00] stopping event stream following
led" module=libcontainerd namespace=plugins.moby
ma@LAPTOP-0VBOP51T:~/XV6_dev$

```